

Лабораторна робота №4

Мета роботи

Опанувати написання складних аналітичних запитів (OLAP) у PostgreSQL. Набути навичок використання агрегатних функцій (SUM, AVG, COUNT, MIN, MAX), групування даних (GROUP BY, HAVING), об'єднання таблиць різними типами джоїнів (INNER, LEFT, RIGHT) та використання підзапитів для формування зведеної бізнес-аналітики на основі розробленої бази даних пункту прокату фільмів.

ЧАСТИНА 1. Базова агрегація та групування (Мінімум 4 запити)

Запит 1.1: Загальна статистика каталогу фільмів (MIN, MAX, COUNT) **Опис:** Запит визначає найстаріший та найновіший фільми в базі, а також загальну кількість фільмів, доступних для прокату.

```
SELECT  
  
    MIN(release_year) AS oldest_film_year,  
  
    MAX(release_year) AS newest_film_year,  
  
    COUNT(film_id) AS total_films_in_catalog  
  
FROM FILM;
```

	oldest_film_year integer	newest_film_year integer	total_films_in_catalog bigint
1	1994	2014	5

Запит 1.2: Підрахунок кількості інвентарю для кожного стану (COUNT, GROUP BY)

Опис: Запит групує всі фізичні диски за їх станом (Новий, Добрий, Пошкоджений) і рахує, скільки дисків знаходиться в кожній категорії. Допомогає адміністратору оцінити знос носіїв.

```
SELECT condition AS disk_condition, COUNT(inventory_id) AS total_disks  
  
FROM INVENTORY  
  
GROUP BY condition;
```

	disk_condition disk_condition	total_disks bigint
1	Good	2
2	Damaged	2
3	New	1

Запит 1.3: Середня вартість оренди фільмів за категоріями (AVG, JOIN, GROUP BY)

Опис: Розраховує середню ціну прокату (rental_rate) для кожного жанру. Результат дозволяє зрозуміти, які жанри є найдорожчими.

```
SELECT c.name AS category_name, ROUND(AVG(f.rental_rate), 2) AS avg_rental_price  
  
FROM CATEGORY c
```

```
JOIN FILM_CATEGORY fc ON c.category_id = fc.category_id
```

```
JOIN FILM f ON fc.film_id = f.film_id
```

```
GROUP BY c.name;
```

	category_name character varying (100)	avg_rental_price numeric
1	Comedy	3.50
2	Drama	4.49
3	Sci-Fi	5.66
4	Action	3.99

Запит 1.4: Клієнти, які витратили більше 10 доларів (SUM, GROUP BY, HAVING)

Опис: Агрегує всі платежі по кожному клієнту і за допомогою речення HAVING відфільтровує лише тих, чия загальна сума витрат перевищує \$10 (пошук VIP-клієнтів).

```
SELECT customer_id, SUM(amount) AS total_spent
```

```
FROM PAYMENT
```

```
GROUP BY customer_id
```

```
HAVING SUM(amount) > 10.00;
```

	customer_id integer	total_spent numeric
1	1	19.99

ЧАСТИНА 2. Об'єднання даних (JOIN) (Мінімум 3 запити)

Запит 2.1: Детальний журнал оренди (INNER JOIN)

Опис: Використовує суворий INNER JOIN для зведення даних з чотирьох таблиць. Показує повну картину: який клієнт взяв який фільм і якого числа. Виводяться лише ті записи, де є повний збіг у всіх таблицях.

```
SELECT c.first_name, c.last_name, f.title AS movie_title, r.rental_date
```

```
FROM CUSTOMER c
```

```
INNER JOIN RENTAL r ON c.customer_id = r.customer_id
```

```
INNER JOIN INVENTORY i ON r.inventory_id = i.inventory_id
```

```
INNER JOIN FILM f ON i.film_id = f.film_id;
```

	first_name character varying (100) 🔒	last_name character varying (100) 🔒	movie_title character varying (255) 🔒	rental_date timestamp without time zone 🔒
1	Oleksandr	Kovalenko	The Matrix	2026-05-10 10:00:00
2	Mariia	Shevchenko	Inception	2026-05-12 11:30:00
3	Dmytro	Moroz	The Matrix	2026-05-15 09:00:00
4	Anna	Boyko	Interstellar	2026-05-16 14:00:00
5	Oleksandr	Kovalenko	Pulp Fiction	2026-05-18 16:45:00

Запит 2.2: Перевірка наявності фільмів на складі (LEFT JOIN)

Опис: Використовує LEFT JOIN між таблицями FILM та INVENTORY. Цей запит виведе *всі* фільми з каталогу, навіть якщо їхніх фізичних копій (дисків) наразі немає на складі (для таких фільмів в колонці inventory_id буде значення NULL).

```
SELECT f.title, i.inventory_id, i.condition
FROM FILM f
LEFT JOIN INVENTORY i ON f.film_id = i.film_id;
```

	title character varying (255) 🔒	inventory_id integer 🔒	condition disk_condition 🔒
1	The Matrix	1	New
2	The Matrix	2	Good
3	Interstellar	4	Good
4	Pulp Fiction	5	Damaged
5	Inception	3	Damaged
6	The Hangover	[null]	[null]

Запит 2.3: Аналіз роботи персоналу (RIGHT JOIN)

Опис: Використовує RIGHT JOIN, де головною таблицею виступає STAFF. Запит покаже всіх співробітників компанії, навіть тих, хто ще не оформив жодної транзакції оренди (наприклад, менеджери або працівники у відпустці).

```
SELECT s.first_name, s.last_name, s.position, r.rental_id, r.rental_date
FROM RENTAL r
RIGHT JOIN STAFF s ON r.staff_id = s.staff_id;
```

	first_name character varying (100) 🔒	last_name character varying (100) 🔒	position staff_position 🔒	rental_id integer 🔒	rental_date timestamp without time zone 🔒
1	Olena	Lysenko	Cashier	1	2026-05-10 10:00:00
2	Olena	Lysenko	Cashier	2	2026-05-12 11:30:00
3	Serhiy	Kravets	Cashier	3	2026-05-15 09:00:00
4	Andriy	Chorny	Cashier	4	2026-05-16 14:00:00
5	Andriy	Chorny	Cashier	5	2026-05-18 16:45:00
6	Svitlana	Melnyk	Manager	[null]	[null]
7	Ivan	Petrenko	Administrator	[null]	[null]

ЧАСТИНА 3. Використання підзапитів (Мінімум 3 запити)

Запит 3.1: Пошук дорогих фільмів (Підзапит у WHERE)

Опис: Знаходить фільми, вартість прокату яких є вищою, ніж середня вартість прокату по всьому каталогу. Підзапит динамічно обчислює середню ціну, з якою порівнюється кожен фільм.

```
SELECT title, rental_rate
```

```
FROM FILM
```

```
WHERE rental_rate > (SELECT AVG(rental_rate) FROM FILM);
```

	title character varying (255) 🔒	rental_rate numeric (5,2) 🔒
1	The Matrix	4.99
2	Inception	5.99
3	Interstellar	5.99

Запит 3.2: Сума витрат клієнта поруч з його іменем (Підзапит у SELECT)

Опис: Виводить імена всіх клієнтів, а в окремій колонці виконує підзапит для розрахунку загальної суми, яку цей клієнт заплатив системі. Це дозволяє уникнути складних джоїнів і групувань.

```
SELECT first_name, last_name, email,
```

```
(SELECT SUM(amount)
```

```
FROM PAYMENT p
```

```
WHERE p.customer_id = c.customer_id) AS total_payments
```

```
FROM CUSTOMER c;
```

	first_name character varying (100) 🔒	last_name character varying (100) 🔒	email character varying (150) 🔒	total_payments numeric 🔒
1	Oleksandr	Kovalenko	alex.koval@kpi.ua	19.99
2	Mariia	Shevchenko	masha.sheva@gmail.co...	5.99
3	Dmytro	Moroz	dima.moroz@ukr.net	5.99
4	Anna	Boyko	anna.boyko@outlook.c...	3.50
5	Artem	Tkachenko	artem.tkach@kpi.ua	[null]

Запит 3.3: Чеки, сума яких вища за середню транзакцію (Підзапит у HAVING)

Опис: Запит групує платежі за статусом і залишає лише ті групи, де середня сума платежу більша, ніж загальна середня сума абсолютно всіх транзакцій у базі даних.

```
SELECT status, ROUND(AVG(amount), 2) AS avg_amount_for_status
```

```
FROM PAYMENT
```

```
GROUP BY status
```

HAVING AVG(amount) > (SELECT AVG(amount) FROM PAYMENT);

	status payment_status 	avg_amount_for_status numeric 
1	Delayed	15.00

Висновок

Під час виконання лабораторної роботи №4 було успішно відпрацьовано навички написання аналітичних OLAP-запитів. За допомогою агрегатних функцій та конструкцій GROUP BY і HAVING було отримано зведену бізнес-статистику (кількість інвентарю, середні ціни, прибуток від клієнтів). Використання різних типів з'єднань (INNER, LEFT, RIGHT) дозволило коректно об'єднувати пов'язані сутності та виявляти "порожні" зв'язки (фільми без дисків, співробітники без продажів). Практика використання підзапитів довела гнучкість SQL у вирішенні багатокрокових аналітичних завдань у межах одного запиту.